

Ostbayerische Technische Hochschule Amberg/Weiden
Department of Electrical Engineering, Media, and
Computer Science

Prof. Dr. Ulrich Schäfer

Project Mobile & Ubiquitous Computing Summer
Semester 2026

Group 03: Simon Völkl & Johannes Schieder

Courses of study: Artificial Intelligence, Industry-4.0-Informatics

June 28, 2026

Contents

1	Introduction	3
1.1	Introduction	3
1.2	Mission Statement	3
1.3	Motivation	3
1.4	Document Structure	3
2	Project Management	3
2.1	SMART Goals	3
2.2	Project Planning	3
2.3	Task Distribution	3
3	Related Work	3
4	Technical Concept	3
4.1	High-Level Architecture	3
4.2	Components	3
5	Hardware	3
5.1	ESP32-S3 Microcontroller	3

5.2	Concurrency	4
5.3	Sensor Data Acquisition	4
5.4	Physics Simulation & Game Logic	4
5.5	Display	4
5.6	MQTT Communication	4
6	Node-RED	4
6.1	Workflow	4
6.2	Dashboard & User Input	4
7	Android App	4
7.1	Software Components, Classes, Views	4
7.2	Dashboard & User Input	4
8	Evaluation	4
8.1	Problems & Solutions	4
8.2	Code Coverage	4
9	Summary and Future Work	4
A	Prompts	4
A.1	Logo Generation	4
A.2	Random DFS Maze Generation Algorithm	5
B	ESP32 Code	7

Abstract

1 Introduction

1.1 Introduction

1.2 Mission Statement

1.3 Motivation

1.4 Document Structure

2 Project Management

2.1 SMART Goals

2.2 Project Planning

2.3 Task Distribution

3 Related Work

4 Technical Concept

4.1 High-Level Architecture

4.2 Components

5 Hardware

Written in Arduino. [?]

5.1 ESP32-S3 Microcontroller

ESP32-S3: [?]

5.2 Concurrency

5.3 Sensor Data Acquisition

5.4 Physics Simulation & Game Logic

5.5 Display

5.6 MQTT Communication

6 Node-RED

[?]

6.1 Workflow

6.2 Dashboard & User Input

7 Android App

7.1 Software Components, Classes, Views

7.2 Dashboard & User Input

8 Evaluation

8.1 Problems & Solutions

[?] should not be used anymore. So we decided on using [?] instead for our dashboard.

8.2 Code Coverage

9 Summary and Future Work

A Prompts

A.1 Logo Generation

Prompt:

A professional mobile app logo for a game titled "BiteBound", presented as a clean, centered circular icon on a transparent background. The design is playful, vibrant, and modern vector-style.

The central focus is a delicious, golden-brown chocolate chip cookie (using warm tones of cookie dough, honey yellow, and rich caramel). The cookie features glossy, melting dark chocolate chunks. Poised playfully on or next to the cookie is a vibrant, single glossy red berry character acting as the player element.

The game title text "BiteBound" can but doesn't have to be integrated beautifully into the circular composition using a bold, friendly, and readable typography in a warm milk-cream color.

Style: Clean 2D vector art, smooth gradients, crisp edges, suitable for an App Store icon.

Color Palette: Golden dough, honey yellow, rich caramel, dark cocoa brown, vibrant berry red, and warm milk cream.

Exclusions: No realistic photographic elements, no messy 3D renders, no complex backgrounds.

Answer:

Image

A.2 Random DFS Maze Generation Algorithm

Prompt:

Act as a computer science expert. For an embedded IoT game ("BiteBound"), I need to design a maze generation algorithm for a grid-mapped display space (conceptually 240x280 with variable wall thickness).

Provide an architectural breakdown of a Randomized Depth-First Search (DFS) algorithm with backtracking. Your response must include:

- Definition and core concepts of Randomized DFS in maze generation.
- A step-by-step description of cell visitation and backtracking mechanics.
- Structured, language-agnostic pseudo-code of the generation loop, stack operations, and random neighbor selection.

STRICT REQUIREMENT: Do not generate any functional source code (e.g., C++, Python). Provide only high-level logical pseudo-code and analysis.

Keep your answer concise, technical, and focused on algorithmic design principles.

Answer:

BITEBOUND: RANDOMIZED DFS MAZE GENERATION ARCHITECTURE

1. CORE CONCEPTS & GRID MAPPING

Randomized Depth-First Search (DFS) generates a "perfect maze" (one unique path between any two points, no loops) by computing a spanning tree over a logical grid graph.

To adapt this to a 240x280 pixel space:

- Abstract the pixel canvas into a logical grid where:
- Total Unit Size (U) = Cell Width + Wall Thickness.
- Columns = 240 / U, Rows = 280 / U.

Memory Optimization: In embedded IoT systems, recursion risks call-stack overflow. Use an iterative approach with an explicit array-based stack in static RAM.

Cell State: Track visitation and wall states (North, South, East, West) using bitmasks packed into a single byte per cell.

2. CELL VISITATION & BACKTRACKING MECHANICS

The algorithm carves a path through solid walls using a random walk, backtracking when it encounters a dead end.

Step-by-Step Execution:

- Initialize: Mark all cells as unvisited with all walls intact. Pick a starting cell as `current_cell` and mark it visited.
- Evaluate: Check the four orthogonal neighbors of `current_cell`. Filter out out-of-bounds or already visited cells.
- Advance: If unvisited neighbors exist, select one at random. Push `current_cell` onto the backtrack stack. Break the wall between `current_cell` and the chosen neighbor. Move to the neighbor and mark it visited.
- Backtrack: If no unvisited neighbors exist, pop a cell from the backtrack stack and make it the `current_cell`.
- Terminate: Repeat steps 2-4 until the backtrack stack is empty.

3. LOGICAL PSEUDO-CODE

FUNCTION GenerateMaze(rows, cols):

Create 2D array 'grid' of size [rows][cols] (all visited = false, all walls intact)

Create array-based 'stack'

`current_cell` = [0, 0]

mark `current_cell` as visited

LOOP WHILE true:

`neighbors` = GetUnvisitedNeighbors(`current_cell`, grid, rows, cols)

 IF `neighbors` is not empty THEN

`next_cell` = RandomSelect(`neighbors`)

 PUSH `current_cell` TO stack

 RemoveWallsBetween(`current_cell`, `next_cell`, grid)

`current_cell` = `next_cell`

 mark `current_cell` as visited

```
    ELSE IF stack is not empty THEN
        current_cell = POP from stack
    ELSE
        BREAK LOOP (Maze complete)
    END IF
END LOOP
END FUNCTION

FUNCTION GetUnvisitedNeighbors(cell, grid, rows, cols):
    unvisited = empty list
    directions = [[-1,0], [1,0], [0,-1], [0,1]] // North, South, West, East

    FOR EACH dir IN directions:
        check_row = cell.row + dir[0]
        check_col = cell.col + dir[1]

        IF check_row within bounds AND check_col within bounds THEN
            IF grid[check_row][check_col].visited is false THEN
                ADD [check_row, check_col] TO unvisited
            END IF
        END IF
    END FOR
    RETURN unvisited
END FUNCTION
```

4. EMBEDDED DESIGN TIP

For high performance on low-power displays, update the screen incrementally by drawing the "carved" paths during step 3, rather than redrawing the whole grid every loop cycle.

B ESP32 Code